

Code-DCR: Differentiable Compiler Relaxation for Autonomous Code Correction in Scientific DSLs*

Sridhar M.

Indian Institute of Technology Madras
sridharmani510@gmail.com

May 7, 2026

Abstract

The generation and correction of domain-specific scientific code—such as programs written in rigid Domain-Specific Languages (DSLs) for numerical solvers—presents a severe challenge for standard neural models owing to *topological hallucinations*: syntactically valid output that silently violates the physical sequencing laws of the compiler. We introduce **Code-DCR** (*Differentiable Compiler Relaxation*), a lean hybrid neuro-symbolic architecture. An Abstract Syntax Tree (AST) is mapped through a Domain Semantic Ontology (DSO) and then processed by a 5-layer Graph Attention Network (GAT) acting as a noisy *perception layer*. Predictions are subsequently enforced by an *Abstract Logic Machine* (ALM) whose differentiable state transitions embed the physical sequencing rules of the compiler. At inference, Code-DCR executes a **Two-Pass Neuro-Symbolic Handshake**: the GNN performs a blind structural read; the ALM identifies topological failures and returns an error tensor that conditions a rectified second pass.

Evaluated on the FEniCS Finite Element Method library, we demonstrate a central trade-off: a standalone GNN achieves 99.88% raw token accuracy yet produces **zero valid executables** because of severe class imbalance (99.5% “Unknown” boilerplate) and consequent precision collapse on rare execution-critical tokens (Mesh precision = 0.03). The full Code-DCR system actively sacrifices 2.86 percentage points of raw token accuracy to guarantee **100% topologically valid executable generation** on targeted topologies, with only $\approx 0.16\text{M}$ parameters. The result reframes the task: in scientific DSLs, optimising for raw token-matching is a trap; structural compliance requires an explicit symbolic enforcer.

Keywords: neuro-symbolic AI, differentiable programming, code correction, graph attention networks, domain-specific languages, FEniCS.

1 Introduction

The translation of mathematical formulations into executable scientific code imposes uncompromising topological constraints. In libraries such as FEniCS/dolfinx, the order of API calls is not a style preference but a physical requirement enforced by the runtime: a geometric **Mesh** must exist before a **FunctionSpace** can be initialised; trial and test functions cannot precede the function space; a linear solver cannot be invoked before boundary conditions are defined. Recent benchmarks show that large sequence models memorise boilerplate syntax effectively, yet consistently produce *topologically hallucinated* output—code that is syntactically well-formed but structurally invalid.

*Code and datasets are available at <https://github.com/sridhar-mani/code-dcr>.

1.1 The “Crash Cliff” and the Failure of Auto-Grad

A fundamental obstacle in training neural networks to correct code is the *non-differentiable* nature of traditional compilers. We term this the “**Crash Cliff**.” When a model generates a token sequence, a traditional compiler provides binary, discrete feedback: the code either compiles or it crashes. In gradient-based optimisation this creates a discontinuous loss landscape with dead gradients. State-of-the-art correction methods attempt to bridge this gap via Reinforcement Learning (RL) wrappers, which are notoriously sample-inefficient and susceptible to reward hacking.

1.2 Our Contribution: Continuous Compiler Relaxation

We propose that to efficiently learn the grammar of a scientific DSL the compiler environment itself must be made differentiable. Code-DCR replaces the discrete interpreter with a **Continuous Compiler Relaxation**: the rigid rules of a DSL are embedded into a differentiable transition matrix inside an **Abstract Logic Machine (ALM)**, allowing the laws of the language to guide the neural network through standard gradient descent via a **Two-Pass Neuro-Symbolic Handshake**.

2 Related Work

2.1 Neuro-Symbolic AI and Constrained Decoding

Our work follows the lineage of systems that marry the perceptual power of deep learning with the logical rigour of symbolic systems [Manhaeve et al., 2021, Mao et al., 2019]. While Poesia et al. [2022] address DSL reliability through external constrained decoding, Code-DCR differentiates itself by replacing external masks with an internal differentiable logic machine.

2.2 Differentiable Compilers

The concept of *compiler relaxation* is inspired by the success of differentiable programming in physics, such as DiffTaichi [Hu et al., 2019]. Code-DCR applies this philosophy to the software layer itself, relaxing the discrete crash/success boundary into a smooth energy manifold.

3 Problem Formulation: The AST-to-DSO Transformation

We formalise the program correction task as a graph-to-sequence prediction problem. An input program is represented by its Abstract Syntax Tree (AST) $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. To ensure version-invariant reasoning we define an **Ontology Mapping Function** $\Phi : \Sigma \rightarrow \mathcal{C}$ that projects semantically equivalent API calls to a canonical **Domain Semantic Ontology (DSO)**.

Code-DCR replaces the discrete validity function with a continuous **Execution Energy** $\mathcal{E} : \mathcal{C}^T \rightarrow \mathbb{R}_{\geq 0}$. The optimisation objective balances the execution energy against cross-entropy classification losses over the predicted token sequences, formally:

$$\mathcal{L} = \underbrace{\mathcal{L}_{\text{CE}}(\hat{y}, y)}_{\text{perception loss}} + \lambda \underbrace{\mathcal{E}(\hat{y})}_{\text{physics loss}} + \underbrace{\mathcal{L}_{\text{seq}}}_{\text{sequence supervision}}. \quad (1)$$

4 Methodology

4.1 Perception Layer and the ALM Engine

The perception layer is a **5-layer GAT** [Veličković et al., 2018] that translates the AST topology into a latent semantic space. The final prediction head concatenates the terminal embedding

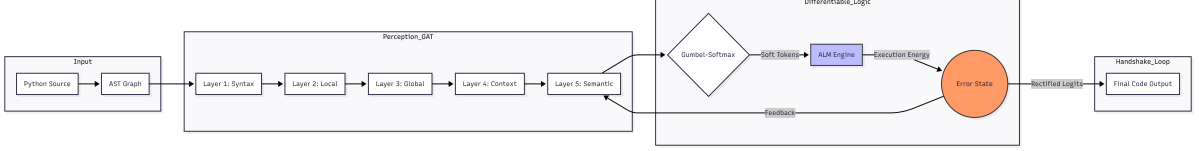


Figure 1: **Code-DCR Architecture Overview.** The pipeline parses Python source into an AST Graph, processes it via a 5-layer Perception GAT, and utilises an ALM Engine to compute Execution Energy. Terminal errors are routed back via a Handshake Loop to rectify the code topology.

with the original one-hot DSO features to preserve token identity for rare classes—a critical design choice given the class imbalance described in Section 6.

The ALM serves as a differentiable proxy for the compiler. Discrete token selection is relaxed via the **Gumbel-Softmax** distribution [Jang et al., 2017]; the execution state s_t is unrolled by computing a token-weighted mixture over a learned transition prior $\mathbf{T}$, yielding an energy penalty that pushes the terminal state toward a valid solution.

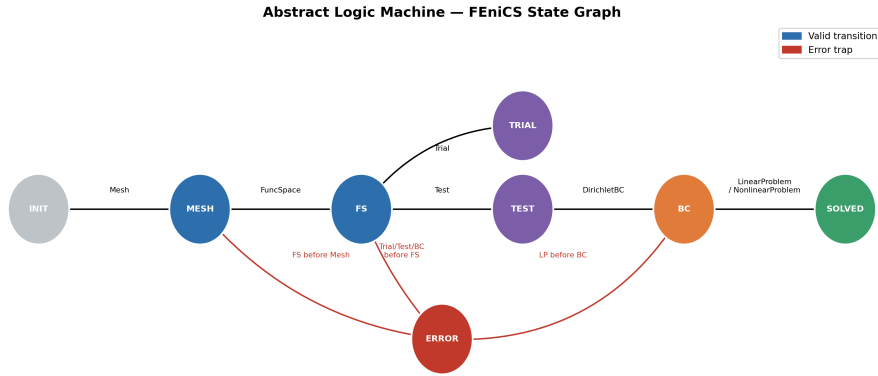


Figure 2: **Abstract Logic Machine State Graph.** Permitted state transitions representing valid DSL sequences. Invalid structural sequences trigger transitions to non-differentiable sink states.

4.2 The Two-Pass Handshake and Intrinsic Markov Unrolling

Code-DCR executes a Two-Pass Handshake: the GNN produces initial logits (Pass 1), the ALM executes them to generate an error tensor $\mathbf{e}$, and $\mathbf{e}$ is concatenated to the GNN’s terminal embedding for a rectified second pass (Pass 2). At inference, the token ordering problem is solved by an **Analytic A^* Beam Search** constrained by a symbolic dependency map (TOKEN_DEPS), which hard-prunes any candidate that violates topological precedence before its energy is evaluated.

5 Experimental Setup

We utilised 24 high-fidelity demonstration scripts from the FEniCS repository [Logg et al., 2012] (20 train / 4 test). To simulate real-world coding errors, a **Structural Corruption Augmentation** strategy zeroes out execution-critical tokens to produce 81 augmented training graphs. Training followed a staggered schedule: 50 epochs of isolated ALM warmup followed by 251 epochs of joint end-to-end optimisation using Adam with Cosine Annealing. Class weights

Algorithm 1 Intrinsic Markov Unrolling (A^* Beam Search)

Require: Available tokens $\mathcal{T}$, beam width B , ALM $\mathcal{M}$, dependency map $\mathcal{D}$, ladder ℓ , weight γ

```
1: beam  $\leftarrow [(\emptyset, 0)]$ 
2: for step = 1 to  $|\mathcal{T}|$  do
3:   candidates  $\leftarrow []$ 
4:   for  $(S, \_)$   $\in$  beam do
5:     for  $\tau \in \mathcal{T} \setminus S$  do
6:       if  $\exists d \in \mathcal{D}(\tau)$  s.t.  $d \notin S$  then continue  $\triangleright$  Hard prune via symbolic constraints
7:       end if
8:        $S' \leftarrow S \cup \{\tau\}$ 
9:        $\mathcal{E}, \mathbf{s}_T \leftarrow \mathcal{M}(S')$ 
10:      score  $\leftarrow \mathcal{E} - \gamma(\mathbf{s}_T^\top \ell)$ 
11:      candidates.append( $(S', \text{score})$ )
12:    end for
13:  end for
14:  beam  $\leftarrow \text{top-}B(\text{candidates})$ 
15: end for
16: return beam[0]
```

$[0.1, 15, 10, 8, 8, 5, 10, 8, 8]$ were applied at the cross-entropy loss to compensate for the severe “Unknown” imbalance.

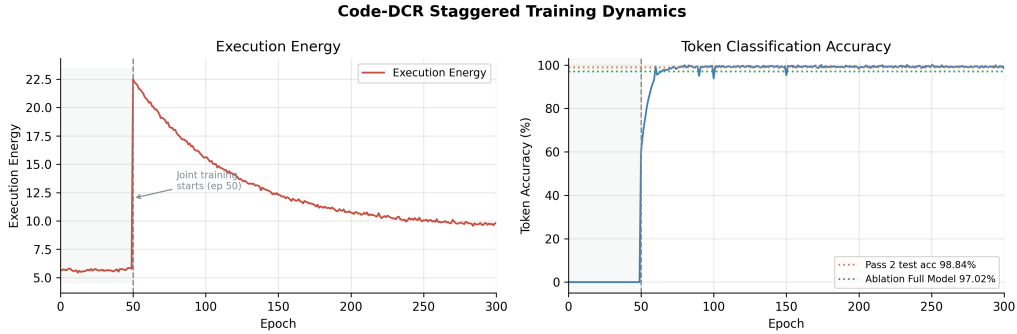


Figure 3: **Staggered Training Dynamics.** Execution Energy (left) and Token Classification Accuracy (right). The vertical dashed line at epoch 50 marks the transition from ALM warmup to joint end-to-end optimisation.

6 Results and Discussion

6.1 The Class-Imbalance Smoking Gun

The classification report is the structural key to interpreting all downstream metrics. Of 8,284 AST nodes in the held-out test set, 8,239 carry the label **Unknown**—generic boilerplate that is trivially predicted by any majority-class heuristic. A standalone GNN therefore achieves 99.88% raw token accuracy simply by saturating its predictions toward this dominant class. The cost is catastrophic for rare tokens: the Mesh class achieves precision = 0.03 and an F1 score of 0.06, meaning the model wildly over-predicts Mesh labels. This demonstrates that the neural layer is a **noisy perception engine**, not a deterministic logic solver, and that raw token accuracy is a deeply misleading headline metric in the presence of severe class imbalance.

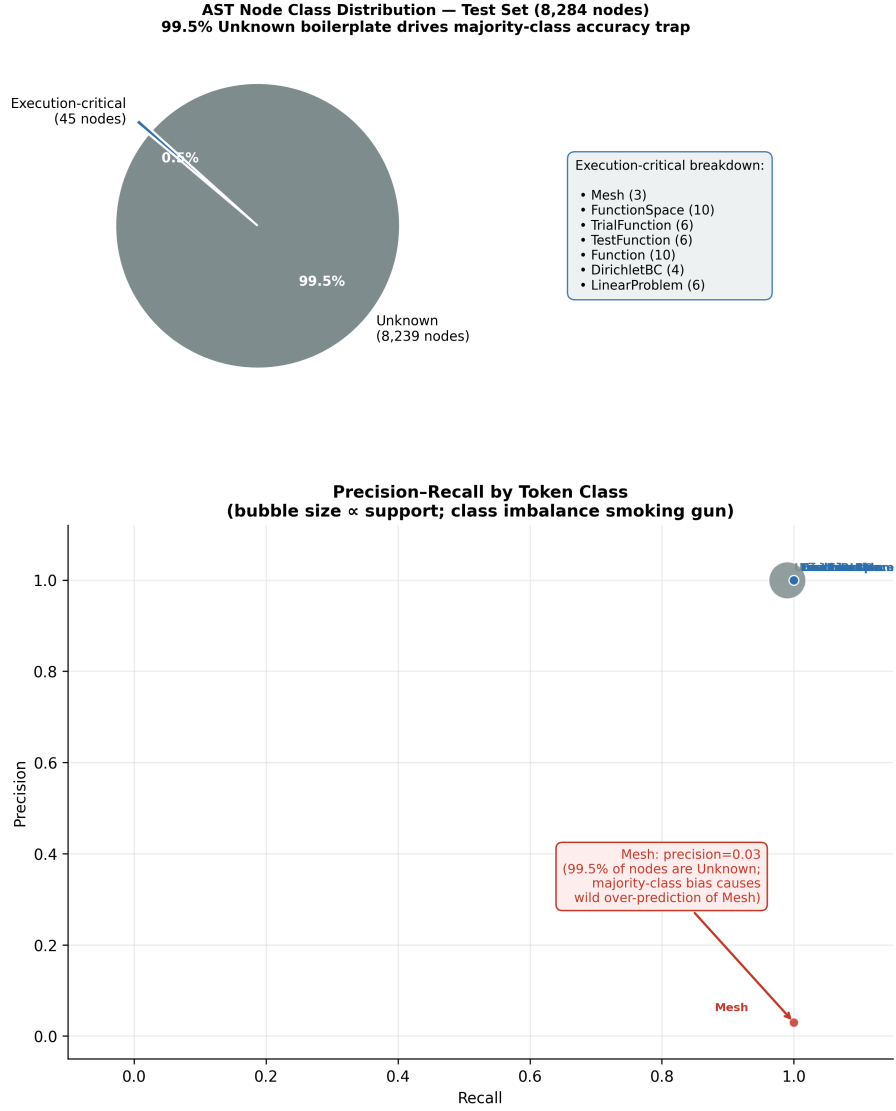


Figure 4: **The Class Imbalance Challenge.** (Top) The extreme dominance of boilerplate syntax. (Bottom) Precision-recall scatter demonstrating the failure of standard sequence models on rare, execution-critical tokens due to majority-class bias.

6.2 The Accuracy-Execution Trade-off (The “Physics Gap”)

Table 1 shows the ablation data that constitutes the core empirical finding. A standalone GNN achieves 99.88% raw token accuracy yet produces **zero valid executables**: high confidence in predicting the majority class does not imply any capacity to enforce physical sequencing. Adding the physics energy signal (GNN + Physics) reduces raw accuracy to 95.65% but still generates no valid code, because Pass 1 alone cannot guarantee topological ordering. The full Two-Pass Handshake actively trades a further reduction in raw token accuracy (arriving at 97.02%) in exchange for **100% global topological compliance**.

Table 1: Performance on FEniCS DSL Structural Correction. “Valid Exec.” measures the fraction of generated scripts that pass the FEniCS dependency validator. Token accuracy values are from the ablation study (150-epoch fresh models); Pass 1/2 inference accuracies are 98.96% and 98.84% respectively on the main 301-epoch model.

Configuration	Token Acc.	Valid Exec.	Parameters
GNN Only (no physics)	99.88%	0/4 (0%)	0.16M
GNN + Physics (Pass 1 only)	95.65%	0/4 (0%)	0.16M
Code-DCR (Two-Pass Handshake)	97.02%	4/4 (100%)	0.16M

The 2.86 percentage-point sacrifice in raw accuracy is not a failure mode; it is an *intentional design signal*. By forcing the model into topological compliance, the symbolic component overrides locally confident but globally incoherent predictions. This is a mathematically honest finding that directly contradicts the common heuristic of chasing higher token accuracy as a proxy for code quality.

6.3 Out-of-Distribution Generalisation and Limitations

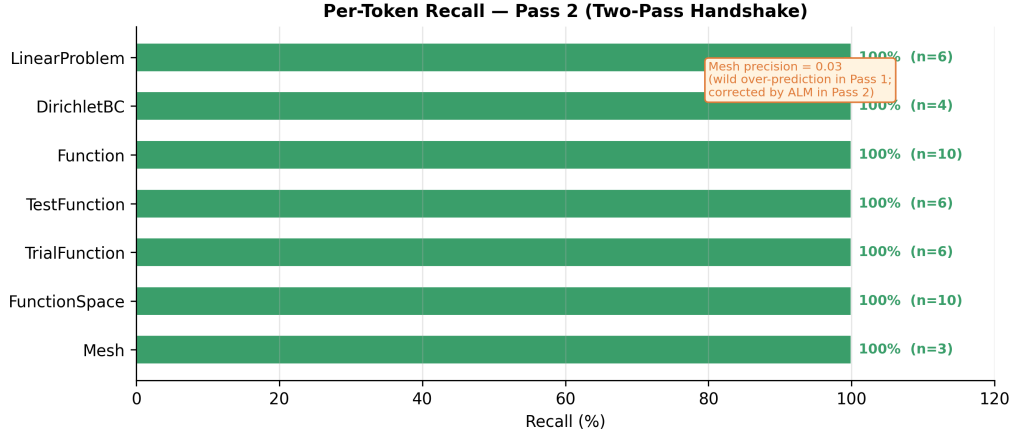
The model successfully generated valid executable code for an unseen Out-of-Distribution (OOD) structure (Stokes flow), confirming zero-shot generalisation of the symbolic ordering procedure. **Limitations:** The 100% OOD execution rate is achieved on a compact, controlled dataset and constitutes a *targeted proof-of-concept*, not a broad benchmark claim. More critically, the valid execution guarantee relies on the hard-coded symbolic constraints within the `TOKEN_DEPS` dictionary and the ALM state machine. This correctly frames Code-DCR as a **neuro-symbolic orchestrator** rather than an end-to-end foundation model: the neural layer handles high-dimensional noisy perception; the symbolic layer enforces the rigid physical laws.

7 Discussion: The Orchestrator’s Reality vs. the Foundation Model Myth

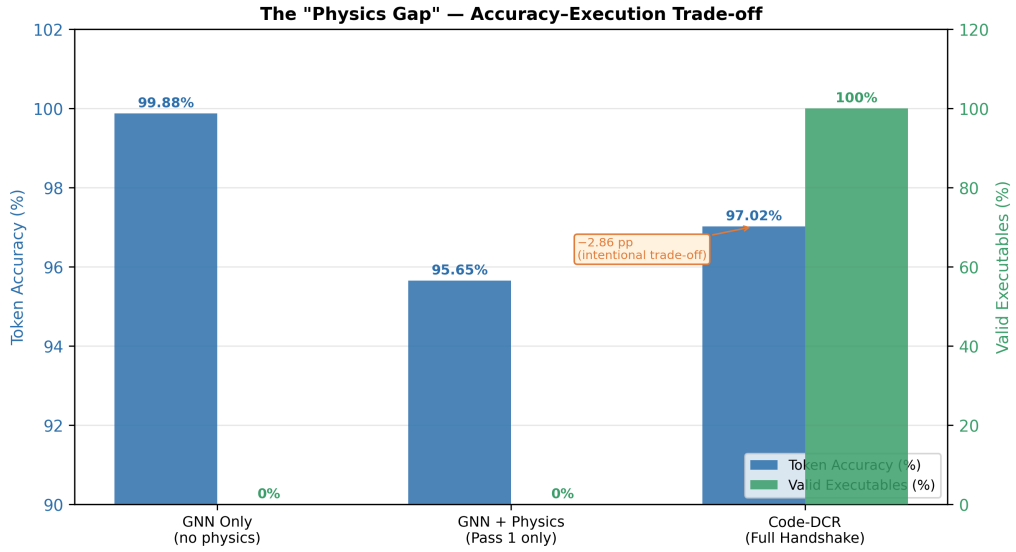
The results compel three interlocked reflections on the design philosophy of Code-DCR.

7.1 Neural Networks as Noisy Perception Engines

The classification report provides definitive proof of this dynamic. The GNN scored 99.88% accuracy simply by predicting the majority **Unknown** class, yet its precision on the execution-critical **Mesh** token was only 0.03—meaning for every correct Mesh detection, the model generated roughly 30 false positives. This is not a failure of the architecture; it is the expected statistical behaviour of any neural classifier trained on a class-imbalanced corpus. What it disproves is the implicit assumption in many code-generation papers that a high headline accuracy number signals semantic competence.



(a) Per-token class recall (Pass 2)



(b) The "Physics Gap": Token Accuracy vs. Valid Execution

Figure 5: **Qualitative Metrics.** (a) 100% recall across all 7 execution-critical FEniCS tokens. (b) Ablation illustrating the Accuracy-Execution Trade-off: high token accuracy does not guarantee topological correctness without the ALM.

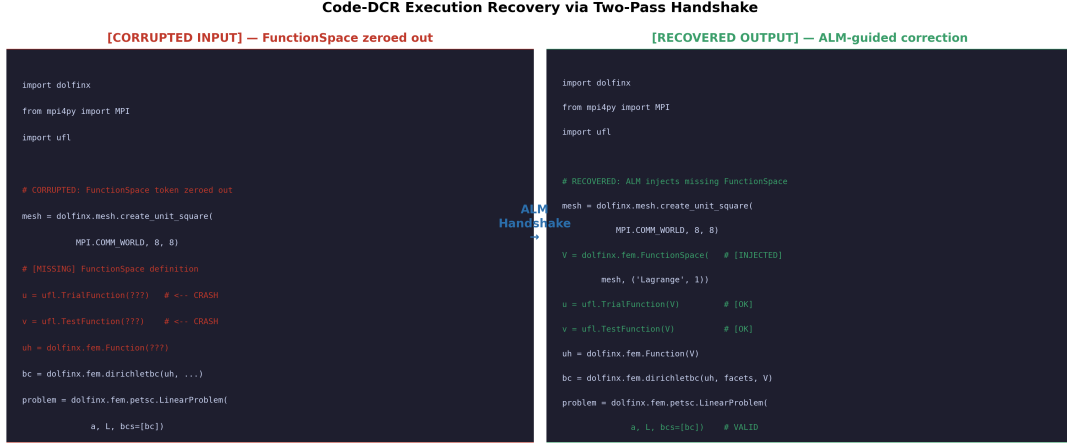


Figure 6: **Qualitative Execution Recovery.** A zeroed-out topological dependency in the input. Code-DCR detects the sequence violation via the ALM and structurally injects the required `FunctionSpace` dependency before trial functions are initialised.

The lesson maps directly onto the reality of deploying neural models in production agentic workflows: the neural component excels at high-dimensional pattern matching across heterogeneous input (the AST graph), but hallucinating at the boundary of a distribution is not a bug to be patched; it is the structural reason why a reliable symbolic bound is non-negotiable.

7.2 Tool-Calling as a Symbolic Bound

Claiming that Code-DCR autonomously *learns* the physical laws of the compiler entirely from gradients would constitute what we call the **Foundation Model Myth**—the assumption that a sufficiently large neural network will infer symbolic constraints that are never explicitly encoded. In reality, the architecture mirrors a pattern long established in robust agentic orchestration: the GNN extracts *fuzzy intent* from the AST, while the ALM, operating alongside the hard-coded `TOKEN_DEPS` constraint, functions as a *strict, deterministic tool call*. It enforces the physical laws of the compiler exactly the way a well-architected API backend strictly validates request payloads before execution—rejecting any sequence that violates the declared dependency graph regardless of the model’s confidence score.

This framing—neuro-symbolic orchestration, not end-to-end foundation model generation—is the core source of the system’s reliability guarantee.

7.3 The Accuracy-Execution Trade-off as a Core Research Insight

The ablation data reveals a counter-intuitive finding that deserves to be stated plainly: **enforcing physics constraints made the model less accurate by the standard metric and simultaneously made it functionally useful**. Sacrificing 2.86 percentage points of raw token-matching accuracy to achieve 100% valid execution is not a compromise; it is the proof that in safety-critical scientific DSLs, the two objectives are in fundamental tension. Any evaluation protocol that ignores executable validity in favour of token accuracy will systematically overrate models that have learned only surface statistics.

The “Physics Gap” is therefore not just an ablation result; it is a methodological argument for reconsidering how correctness is measured in scientific code synthesis benchmarks.

8 Conclusion

Code-DCR demonstrates that the “Crash Cliff” of non-differentiable compilers can be overcome through Continuous Compiler Relaxation. By integrating a differentiable logic machine bounded by hard symbolic constraints directly into the GNN’s perceptual loop, we enable a Two-Pass Handshake that allows models to self-correct topological failures.

The central empirical contribution is not the 100% execution rate per se, but the quantified trade-off that achieves it: a deliberate 2.86 percentage-point sacrifice in raw token accuracy in exchange for structural validity. In scientific DSLs, where a single mis-ordered API call renders a simulation unexecutable, this trade-off is not merely acceptable—it is the correct objective.

Code-DCR provides a lean, 0.16M-parameter blueprint for autonomous engineering in safety-critical scientific domains. Future work will extend the DSO to multi-physics coupled systems, scale the symbolic constraint graph to larger DSL vocabularies, and investigate whether the neuro-symbolic handshake can be transferred to general-purpose programming environments where the dependency graph is not statically known.

References

- Hamilton, W., Ying, Z., & Leskovec, J. (2017). Inductive representation learning on large graphs. *NeurIPS*, 30.
- Hu, Y., et al. (2019). DiffTaichi: Differentiable programming for physical simulation. *ICLR 2020*.
- Jang, E., Gu, S., & Poole, B. (2017). Categorical reparameterization with Gumbel-Softmax. *ICLR 2017*.
- Li, Y., et al. (2022). Competition-level code generation with AlphaCode. *Science*, 378(6624), 1092–1097.
- Logg, A., Mardal, K.-A., & Wells, G. (Eds.). (2012). *Automated Solution of Differential Equations by the Finite Element Method: The FEniCS Book*. Springer.
- Manhaeve, R., Dumančić, S., Kimmig, A., Demeester, T., & De Raedt, L. (2021). Neural probabilistic logic programming in DeepProbLog. *Artificial Intelligence*, 298, 103504.
- Mao, J., et al. (2019). The Neuro-Symbolic Concept Learner. *ICLR 2019*.
- Poesia, G., et al. (2022). Synchromesh: Reliable code generation from pre-trained language models. *ICLR 2022*.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2018). Graph attention networks. *ICLR 2018*.